
CODEC TECHNOLOGIES

Internship Project Report

Vulnerability Assessment and Penetration Testing (VAPT)

Submitted by:
Vishnu Vardhan

Organization	Codec Technologies
Project Title	Vulnerability Assessment and Penetration Testing (VAPT)
Target Application	DVWA (Damn Vulnerable Web Application)
Security Level Tested	Low

1. Introduction

This report documents the Vulnerability Assessment and Penetration Testing (VAPT) project completed as part of the internship at Codec Technologies. The objective of this project was to identify and exploit common web application vulnerabilities in a controlled, legal environment using industry-standard tools and methodologies.

The project was conducted on the Damn Vulnerable Web Application (DVWA), which is an intentionally insecure PHP/MySQL web application designed for security professionals and students to practice penetration testing skills.

1.1 Project Objectives

- Perform service and port enumeration on the target system using Nmap
- Intercept and analyze HTTP traffic using Burp Suite
- Identify and exploit SQL Injection vulnerability in DVWA
- Identify and exploit Reflected Cross-Site Scripting (XSS) vulnerability in DVWA
- Document findings, severity levels, and generate a professional penetration testing report

1.2 Scope of Testing

The penetration testing was limited strictly to the locally hosted DVWA instance running on the tester's own machine (localhost). No external systems, networks, or third-party assets were targeted. All testing was conducted ethically and within the boundaries of a simulated environment.

1.3 Tools Used

Tool	Version / Type	Purpose
Kali Linux	OS	Operating system and testing platform
Docker	Containerization	Deploy DVWA in isolated container
DVWA	v1.10	Target web application for testing
Nmap	v7.99	Network and port scanning
Burp Suite	Community v2026.3.2	HTTP traffic interception and analysis

2. Environment Setup

2.1 Operating System

OS: Kali Linux (Debian-based)

Kali Linux was used as the primary attack machine for this project. It comes pre-installed with numerous security tools including Nmap, Burp Suite, and Metasploit, making it the ideal platform for penetration testing activities.

2.2 Target Application Deployment using Docker

The Damn Vulnerable Web Application (DVWA) was deployed locally using Docker to provide an isolated and safe testing environment. The following commands were used:

Docker Commands:

```
sudo docker pull vulnerables/web-dvwa
sudo docker run --rm -it -p 80:80 vulnerables/web-dvwa
```

Purpose: To run the DVWA application in a Docker container, exposing it on port 80 of localhost.

2.3 DVWA Access Details

Parameter	Value
Access URL	http://localhost
Username	admin
Password	password
Security Level	Low
PHPIDS	Disabled

2.4 DVWA Homepage Screenshots

After logging in successfully with the admin credentials, the DVWA homepage was accessed. The security level was confirmed as 'Low' to enable all vulnerabilities for testing.

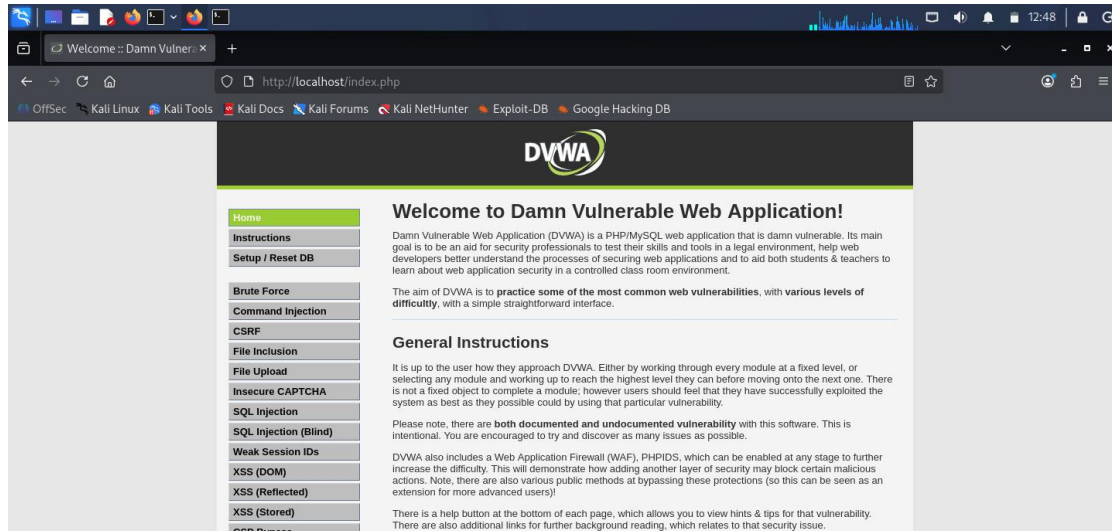


Figure 1: DVWA Homepage — Welcome Page with Vulnerability Menu

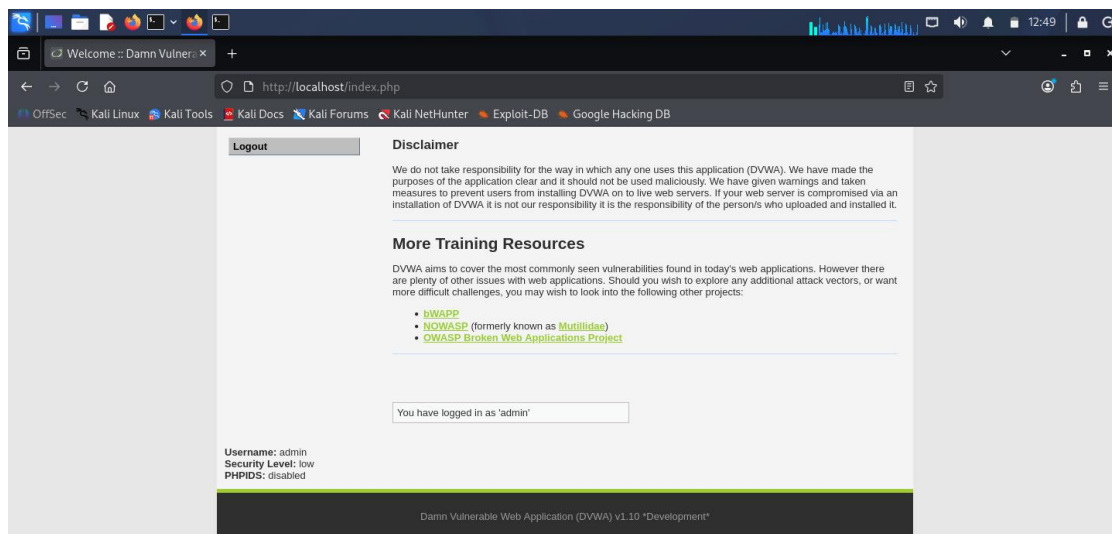


Figure 2: DVWA Homepage — Logged in as 'admin', Security Level: Low

3. Tool 1: Nmap — Network Scanning

3.1 Overview

Nmap (Network Mapper) is a free and open-source network discovery and security auditing utility. In this project, Nmap was used to identify open ports and running services on the localhost target machine.

3.2 Command Used

```
nmap -sV localhost
```

The -sV flag enables version detection, which probes open ports to determine the service name and version running on those ports.

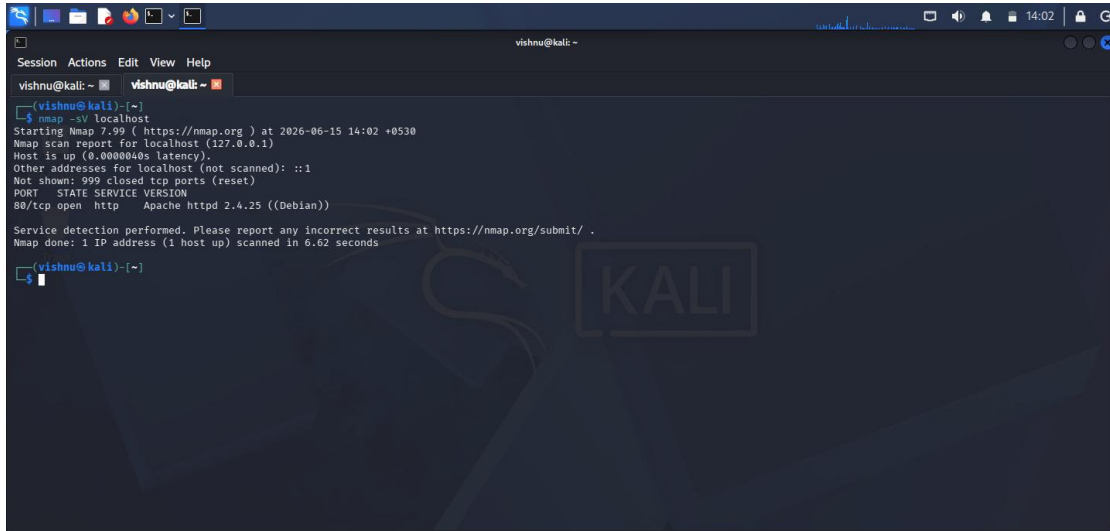
3.3 Scan Results

Port	State	Service	Version
80/tcp	Open	HTTP	Apache httpd 2.4.25 (Debian)

3.4 Observations

- Only port 80 (HTTP) was found open on localhost.
 - Apache HTTP Server version 2.4.25 (Debian) was identified as the running web service.
 - 999 closed TCP ports were detected, indicating a minimal attack surface.
 - The scan completed in 6.62 seconds with 1 IP address (1 host) scanned.
-

3.5 Screenshot



```
vishnu@kali: ~  
$ nmap -sV localhost  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-15 14:02 +0530  
Nmap scan report for localhost (127.0.0.1)  
Host is up (0.0000040s latency).  
Other addresses for localhost (not scanned): ::1  
Not shown: 999 closed tcp ports (reset)  
PORT      STATE SERVICE VERSION  
80/tcp    open  http      Apache httpd 2.4.25 ((Debian))  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 6.62 seconds  
  
vishnu@kali: ~
```

Figure 3: Nmap -sV Scan Result — Port 80 Open, Apache 2.4.25 Detected

4. Tool 2: Burp Suite — HTTP Traffic Analysis

4.1 Overview

Burp Suite Community Edition is a widely used web application security testing platform. It functions as an intercepting proxy between the browser and the web application, allowing the analyst to capture, inspect, and modify HTTP/HTTPS requests and responses in real time.

4.2 Configuration Steps

- Launched Burp Suite Community Edition v2026.3.2.
- Navigated to Proxy → Intercept tab and turned Intercept ON.
- Configured the browser to route traffic through Burp's proxy (127.0.0.1:8080).
- Browsed to `http://localhost` to generate HTTP traffic.
- Observed captured requests in the HTTP History tab.

4.3 HTTP History Observations

The following HTTP requests were captured in Burp Suite's proxy history while interacting with DVWA:

#	URL	Method	Status	Notes
2	/login.php	GET	200	Login page loaded
6	/login.php	POST	302	Login form submitted
7	/index.php	GET	200	Welcome :: Damn Vulnerable...
12	/index.php	GET	200	Index page (subsequent request)

4.4 Intercepted Request Analysis

A GET request to `http://localhost/index.php` was intercepted and analyzed in Burp Suite. Key HTTP headers observed included:

- Host: localhost
 - Cache-Control: max-age=0
 - Accept-Language: en-GB, en; q=0.9
 - User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 Chrome/146
 - Upgrade-Insecure-Requests: 1
 - Request Cookies: 2 (including PHPSESSID session cookie)
-

4.5 Screenshots

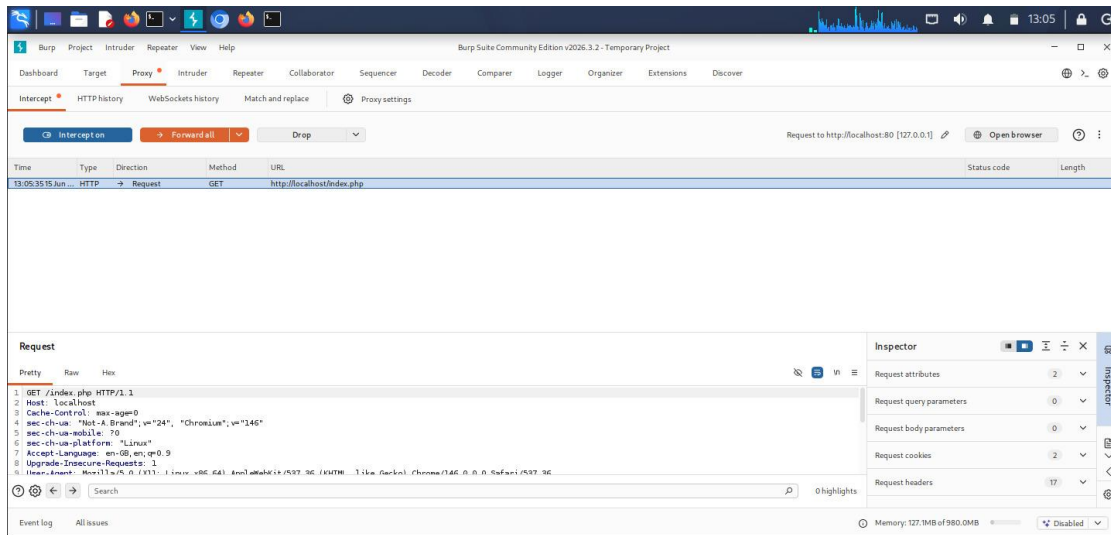


Figure 4: Burp Suite — Proxy Intercept tab with captured GET request to /index.php

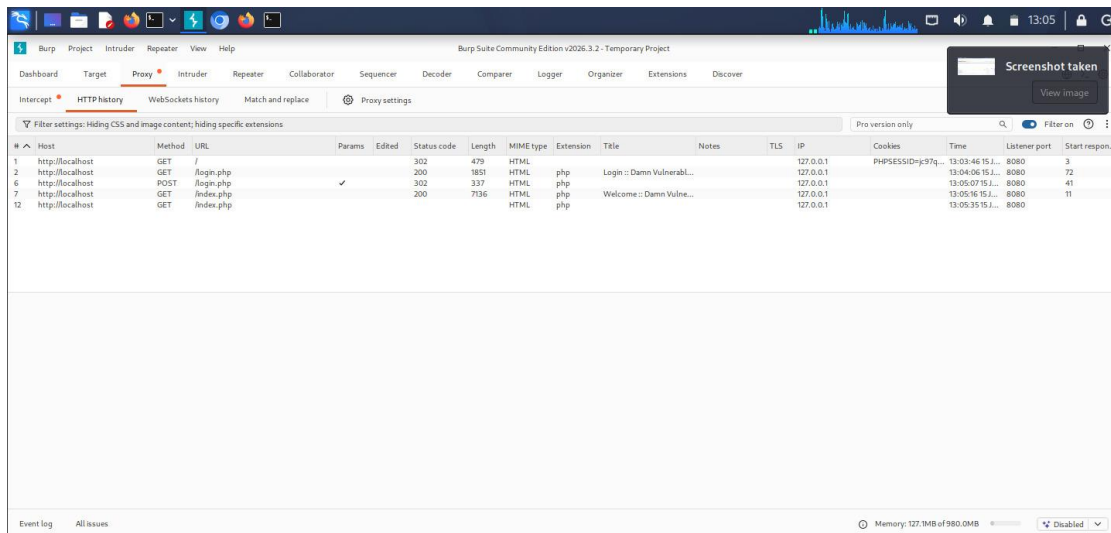


Figure 5: Burp Suite — HTTP History tab showing full request log

5. Vulnerability 1: SQL Injection

5.1 Overview

SQL Injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. It is one of the most critical vulnerabilities listed in the OWASP Top 10 and can lead to unauthorized access to sensitive data, authentication bypass, and in severe cases, complete database compromise.

5.2 Testing Details

DVWA Module	SQL Injection
Test URL	http://localhost/vulnerabilities/sqli/
Input Field	User ID
Payload Used	1' OR '1'='1
Security Level	Low

5.3 Exploitation Steps

- Navigated to the SQL Injection module in DVWA.
- Entered the payload **1' OR '1'='1** in the User ID field.
- Clicked Submit to execute the query.
- The application returned all user records from the database without proper authentication or filtering.

5.4 How the Payload Works

The input **1' OR '1'='1** modifies the backend SQL query logic. The single quote (') closes the original string parameter, and the **OR '1'='1** condition is always true, causing the query to return all rows in the users table instead of filtering by a specific ID.

5.5 Observation and Result

All five user records were returned: admin, Gordon Brown, Hack Me, Pablo Picasso, and Bob Smith. This confirms that the application is vulnerable to SQL Injection and that unauthorized access to the full user database is possible.

5.6 Severity and Impact

Severity	HIGH
Impact	Unauthorized access to all database records
Risk	Data theft, authentication bypass, full DB dump
Remediation	Use parameterized queries / prepared statements

5.7 Screenshots

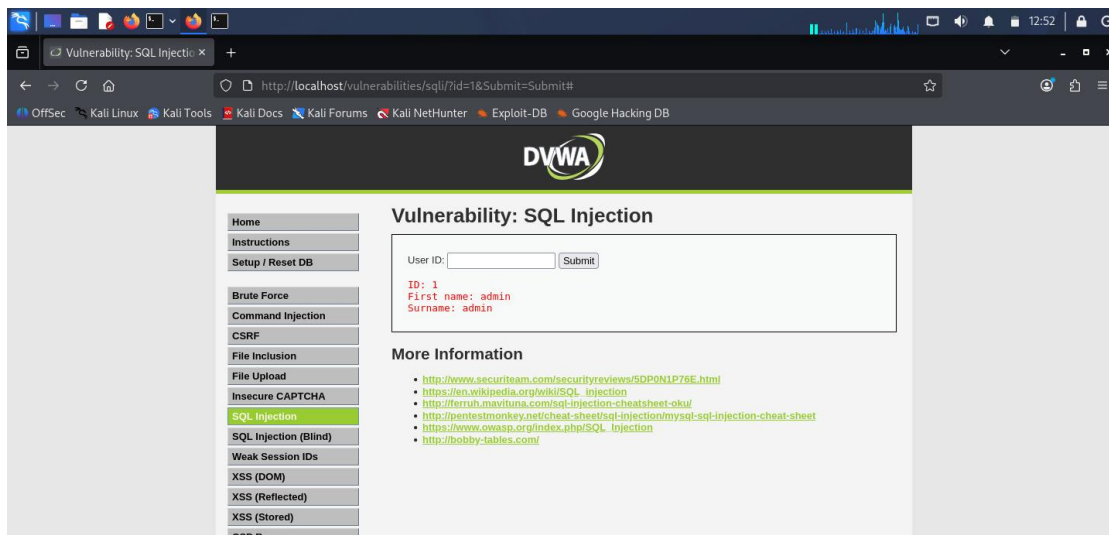


Figure 6: SQL Injection — Normal query with User ID=1, returning admin record

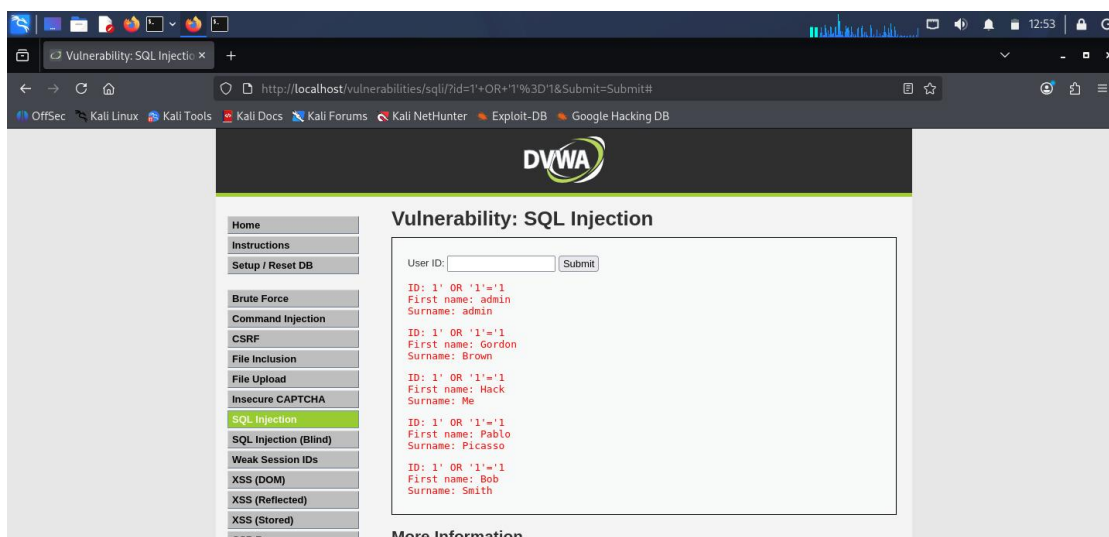


Figure 7: SQL Injection — Payload '1' OR '1'='1' returning all 5 user records

6. Vulnerability 2: Reflected Cross-Site Scripting (XSS)

6.1 Overview

Cross-Site Scripting (XSS) is a client-side code injection vulnerability that allows attackers to inject malicious scripts into web pages viewed by other users. Reflected XSS occurs when user-supplied data is immediately reflected back in the server's response without proper sanitization or encoding. This can lead to session hijacking, credential theft, phishing, and defacement attacks.

6.2 Testing Details

DVWA Module	XSS (Reflected)
Test URL	<code>http://localhost/vulnerabilities/xss_r/</code>
Input Field	What's your name?
Payload 1	<code><h1>vishnu</h1></code>
Payload 2	<code><a href><h1>vishnu<h1></code>
Security Level	Low

6.3 Exploitation Steps

- Navigated to the XSS (Reflected) module in DVWA.
- First, entered a plain name 'vishnu' to observe the normal output (Hello vishnu displayed in red text).
- Then entered the HTML payload `<h1>vishnu</h1>` in the name field and submitted.
- The browser rendered the input as an H1 heading, confirming HTML injection.
- Then tested a more complex payload `<a href><h1>vishnu<h1></a>` to confirm hyperlink injection as well.
- In both cases, user-supplied HTML was rendered directly in the page, confirming Reflected XSS.

6.4 Observation and Result

The application reflected the user input without any encoding or sanitization. When the HTML payload was submitted, the browser rendered it as actual HTML content. The text 'vishnu' appeared as a large heading (H1), and when the anchor tag payload was used, it rendered as a clickable hyperlink. This confirms the presence of a Reflected XSS vulnerability.

6.5 Severity and Impact

Severity	MEDIUM
Impact	HTML/script injection in victim's browser
Risk	Session hijacking, phishing, page defacement
Remediation	HTML encode output; implement Content Security Policy (CSP)

6.6 Screenshots

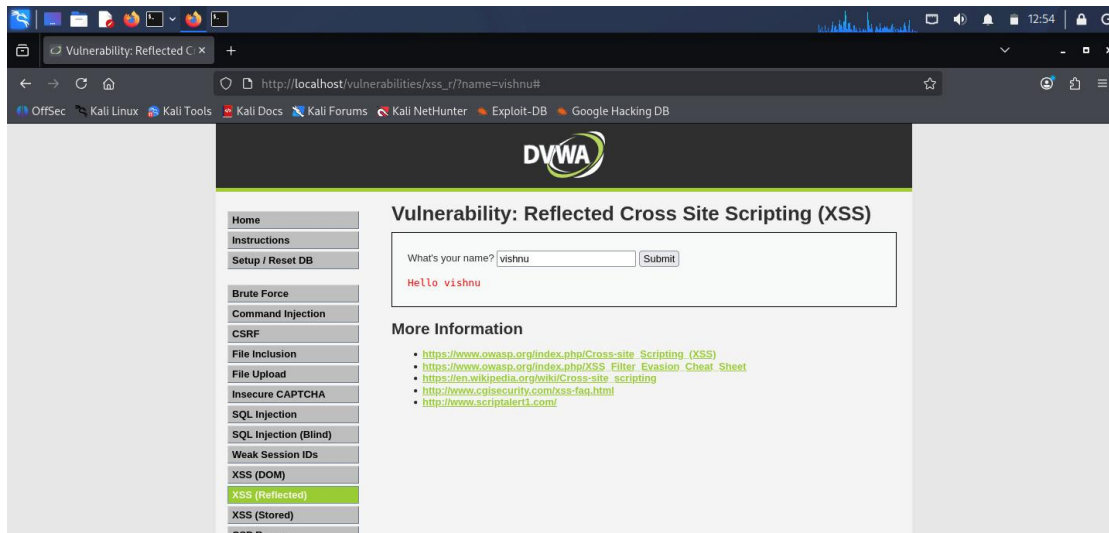


Figure 8: Reflected XSS — Normal input 'vishnu' reflected as plain text

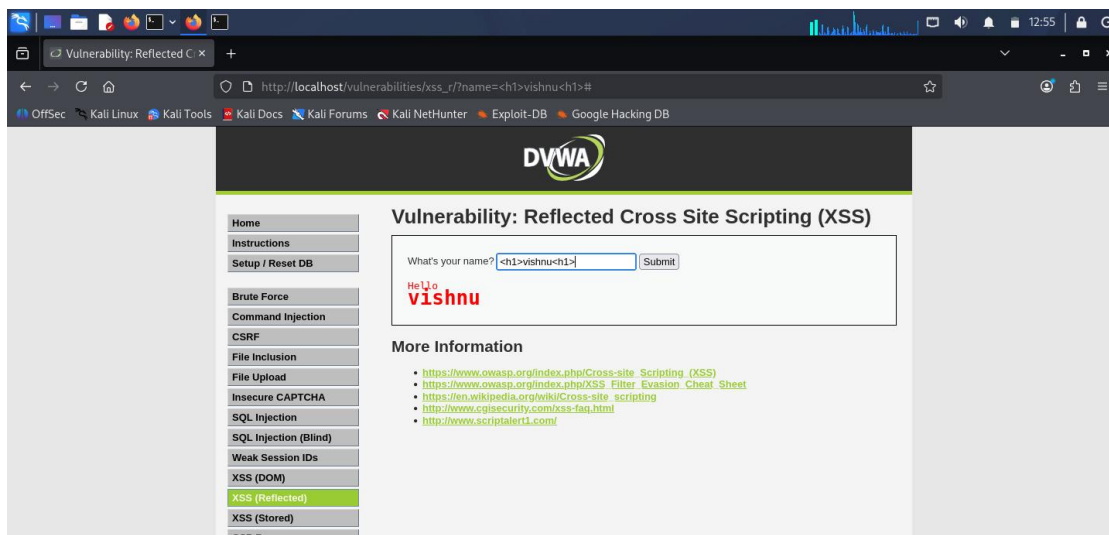


Figure 9: Reflected XSS — Payload <h1>vishnu</h1> rendered as H1 heading

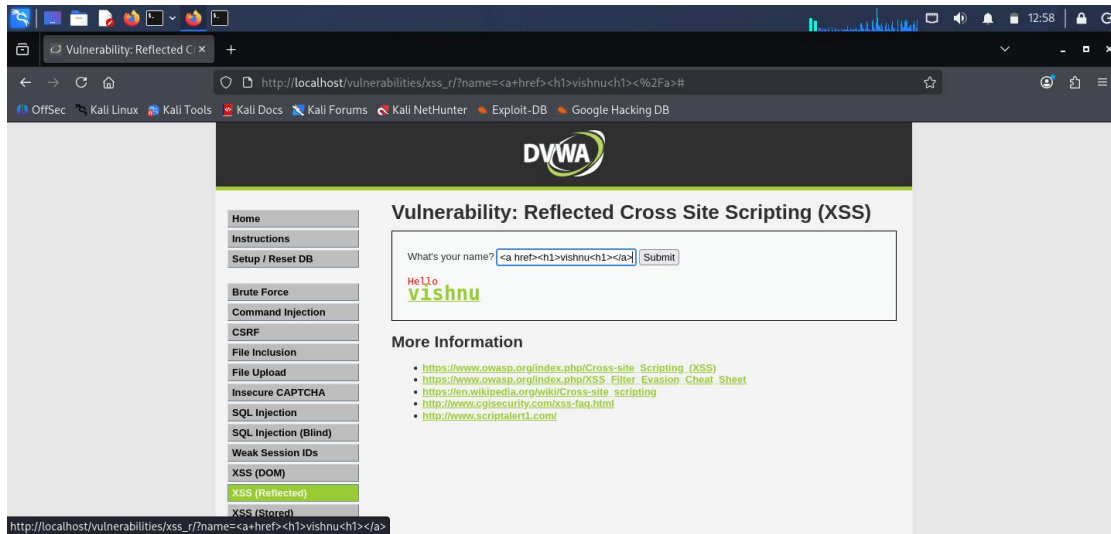


Figure 10: Reflected XSS — Anchor tag payload rendered as hyperlink in the page

7. Findings Summary

7.1 Vulnerability Summary Table

Finding	Description	Severity	Tool Used
Service Enumeration	Port 80 open; Apache 2.4.25 identified	LOW	Nmap
HTTP Traffic Exposure	HTTP requests captured and analyzed	INFORMATIONAL	Burp Suite
SQL Injection	All DB records exposed via boolean payload	HIGH	DVWA / Manual
Reflected XSS	HTML rendered from unencoded user input	MEDIUM	DVWA / Manual

7.2 Risk Rating Summary

Severity Level	Count	Vulnerabilities
Critical	0	None
High	1	SQL Injection
Medium	1	Reflected XSS
Low	1	Service Enumeration
Informational	1	HTTP Traffic Analysis

8. Skills Learned

The following technical skills and competencies were developed and demonstrated through the completion of this project:

Skill Area	Description
Kali Linux Basics	Navigating the Kali Linux operating system, using terminals, and managing tools.
Docker Deployment	Pulling and running Docker containers; deploying DVWA with docker run.
Nmap Scanning	Performing service/version detection scans to enumerate open ports and services.
HTTP Request Analysis	Understanding HTTP methods, status codes, headers, and cookies.
Burp Suite Usage	Configuring Burp proxy, intercepting live requests, reviewing HTTP history.
SQL Injection Testing	Crafting and executing boolean-based SQL payloads to bypass query logic.
XSS Testing	Identifying reflected XSS by injecting and observing rendered HTML in responses.
Vulnerability Reporting	Documenting findings with severity ratings, impact analysis, and remediation advice.

9. Conclusion

This internship project at Codec Technologies provided valuable hands-on experience in the field of web application security and penetration testing. By deploying and testing the Damn Vulnerable Web Application (DVWA) in a safe, controlled Docker environment on Kali Linux, the following key outcomes were achieved:

- Successfully performed network reconnaissance using Nmap, discovering the Apache HTTP service running on port 80.
- Effectively used Burp Suite Community Edition to intercept, inspect, and analyze live HTTP traffic between the browser and the DVWA application.
- Exploited a High-severity SQL Injection vulnerability using the payload `1' OR '1'='1`, resulting in full disclosure of all database user records.
- Exploited a Medium-severity Reflected XSS vulnerability by injecting HTML tags into a user input field, confirming that user-supplied content was rendered unsanitized in the browser.

These exercises reinforced the importance of secure coding practices such as parameterized queries, input validation, and output encoding. The project also demonstrated the critical role that penetration testing plays in identifying and remediating vulnerabilities before malicious actors can exploit them.

The knowledge gained through this project — including the use of industry-standard tools like Nmap and Burp Suite, an understanding of OWASP Top 10 vulnerabilities, and the ability to document security findings professionally — forms a strong foundation for a career in cybersecurity and ethical hacking.

10. References

- OWASP Top 10 Web Application Security Risks — <https://owasp.org/www-project-top-ten/>
 - DVWA Official Repository — <https://github.com/digininja/DVWA>
 - Nmap Official Documentation — <https://nmap.org/docs.html>
 - Burp Suite Documentation — <https://portswigger.net/burp/documentation>
 - OWASP XSS Prevention Cheat Sheet —
https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
 - OWASP SQL Injection Prevention Cheat Sheet —
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
-